

Double-Buffered SIGReg

Working Paper

Aurélien Cecille

May 14, 2026

Abstract

Self-supervised learning (SSL) architectures that rely on distribution matching, such as those utilizing the Epps-Pulley empirical characteristic function test, enforce regularization by shaping the latent space into an isotropic Gaussian. A major bottleneck of these approaches is the requirement for massive batch sizes to accurately estimate collective distribution statistics, leading to heavy memory and computational constraints. While temporal accumulation of characteristic-function statistics can theoretically decouple the statistical estimate from the instantaneous mini-batch size, it fails in practice when random projection axes must be refreshed to avoid dimensional collapse in high-dimensional spaces (the “Null Space Trap”). In this paper, we propose Double-Buffered SIGReg (DB-SIGReg), a random projection mechanism that maintains a mature active projection buffer for gradient computation and a shadow buffer for silent statistical warm-up. We further show that treating the accumulated global distribution estimate as a detached constant factors the squared batch error into a linear, independent pseudo-loss per sample. Combined with gradient accumulation, this formulation approximates the stability of massive batch-size regularization while operating under single-device memory constraints.

1 Introduction

Modern representation learning heavily relies on preventing representation collapse. While contrastive methods explicitly repel negative samples, regularization-based methods enforce structural properties on the latent space. Recent advances, such as Joint-Embedding Predictive Architectures regularized via the Epps-Pulley test or Sketched Isotropic Gaussian Regularization (SIGReg), force the representation manifold to match a target isotropic Gaussian distribution.

A fundamental limitation of matching distributions is that a distribution is a collective property of a batch, not an individual sample. To prevent highly volatile gradient updates driven by noisy empirical means, these architectures require exceptionally large batch sizes (e.g., $B \geq 4096$). This imposes a massive memory footprint, as the intermediate activations of the backbone encoder (typically a Vision Transformer) scale quadratically with sequence length and linearly with batch size.

To alleviate this, one might naturally attempt to accumulate the cosine and sine sufficient statistics of the characteristic function across several mini-batches, then normalize those accumulated sums into a large-batch estimate. However, computing the distribution distance in high-dimensional Euclidean space is intractable, necessitating the projection of embeddings onto random 1D axes. The inherent contradiction arises here: to prevent the network from hiding collapsed representations in the null space of the projections, the random axes must be frequently re-sampled. Yet, if the axes change, the accumulated statistics are tied to the old projection frame and become instantly invalid.

Temporal accumulation also exposes a second difficulty: the squared error of the Empirical Characteristic Function (ECF) intrinsically links the gradient of one sample to the coordinates of all other samples in the batch. This cross-term dependency prevents naive gradient accumulation across mini-batches from exactly reproducing the behavior of a massive batch.

We introduce **Double-Buffered Random Projections**, a mechanism inspired by computer graphics and systems engineering, to resolve these contradictions. By maintaining two overlap-

ping states—an active projection matrix providing stable gradients and a shadow projection matrix accumulating valid statistics in the background—we seamlessly rotate the geometric evaluation frame without invalidating the statistics needed by the next frame. We then derive an independent sample pseudo-loss that uses the mature accumulated characteristic-function estimate as a detached global constant, enabling memory-efficient gradient accumulation.

2 Preliminaries

2.1 The Characteristic Function Objective

The Epps-Pulley test operates in the frequency domain, measuring the distance between the Empirical Characteristic Function (ECF) of the batch and the theoretical characteristic function of a standard normal distribution, $\phi(t) = \exp(-t^2/2)$. For a set of 1D scalar projections z , the test evaluates:

$$\mathcal{L}_{EP} = \sum_{k=1}^T w_k \left| \frac{1}{B} \sum_{j=1}^B e^{it_k z_j} - e^{-t_k^2/2} \right|^2 \quad (1)$$

where t_k and w_k are fixed quadrature points and weights derived from Gauss-Hermite integration. The loss naturally decouples into cosine and sine terms:

$$\mathcal{L}_{EP} = \sum_{k=1}^T w_k \left[\left(\frac{1}{B} \sum_{j=1}^B \cos(t_k z_j) - \phi(t_k) \right)^2 + \left(\frac{1}{B} \sum_{j=1}^B \sin(t_k z_j) \right)^2 \right]. \quad (2)$$

2.2 The Random Projection Contradiction

To apply this to a D -dimensional latent space, the representations $Z \in \mathbb{R}^{B \times D}$ are projected onto a matrix of A random directions, $P \in \mathbb{R}^{D \times A}$, yielding $Z_{proj} = ZP$. According to the Cramér-Wold theorem, Z is Gaussian if and only if all its 1D projections are Gaussian.

- **If P is fixed forever:** The neural network learns to map unregularized, collapsed structures into the null space of P . The 1D projections appear Gaussian, but the D -dimensional space is severely distorted.
- **If P is re-sampled at every step:** We prevent the null space trap, but we cannot accumulate $\cos(t \cdot ZP)$ or $\sin(t \cdot ZP)$ statistics over time, as the reference frame P constantly shifts, forcing us to rely on the current batch size B for the ECF estimate.

2.3 The Cross-Term Dependency

The batch mean inside $\mathcal{L}_{EP}$ makes the gradient of sample z_i depend on every sample in the batch. For the cosine term at frequency t_k , define:

$$\mu_{\cos,k} = \frac{1}{B} \sum_{j=1}^B \cos(t_k z_j). \quad (3)$$

Then:

$$\frac{\partial}{\partial z_i} (\mu_{\cos,k} - \phi(t_k))^2 = \frac{2}{B} (\mu_{\cos,k} - \phi(t_k)) (-t_k \sin(t_k z_i)). \quad (4)$$

The error term $\mu_{\cos,k} - \phi(t_k)$ is shared across all samples, so it cannot be computed independently for each mini-batch unless a stable estimate of the global batch mean is available.

3 Method: Double-Buffered SIGReg

To combine the memory efficiency of temporal accumulation with the geometric robustness of moving projections, we decouple the gradient generation step from the statistical accumulation step. We maintain a mature active projection operator for gradient computation and a shadow projection operator that silently feeds a statistic buffer for the next geometric frame.

3.1 The Active and Shadow States

We define a window size of K steps. The system initializes two identical-sized random projection operators:

1. A_{active} : The active projection operator.
2. A_{shadow} : The shadow projection operator.

Each operator maps from the latent dimension to A projected axes. If implemented as dense random matrices, each has shape $D \times A$. This should be distinguished from the statistic buffers, which store only accumulated cosine and sine statistics for each projected axis and frequency.

During steps 1 to K , the network executes the forward pass through both matrices:

$$Z_{active} = Z A_{active}, \quad (5)$$

$$Z_{shadow} = Z A_{shadow}. \quad (6)$$

The active branch participates in backpropagation. The shadow branch is evaluated under `no_grad` and exists only to build valid statistics for the next active frame.

3.2 Independent Sample Pseudo-Loss

Let the mature active statistics $\mathcal{H}_{\cos}^{(active)}(t_k)$ and $\mathcal{H}_{\sin}^{(active)}(t_k)$ denote normalized estimates obtained from accumulated cosine and sine sums in the active projection frame. We detach these estimates from the computational graph and define:

$$C_k = \text{stopgrad} \left(\mathcal{H}_{\cos}^{(active)}(t_k) - \phi(t_k) \right), \quad (7)$$

$$S_k = \text{stopgrad} \left(\mathcal{H}_{\sin}^{(active)}(t_k) \right). \quad (8)$$

Substituting these global constants for the current mini-batch error removes the cross-term dependency. For a projected scalar sample z_i , we use the independent pseudo-loss:

$$\mathcal{L}_{ind}(z_i) = \sum_{k=1}^T w_k [C_k \cos(t_k z_i) + S_k \sin(t_k z_i)]. \quad (9)$$

Its derivative is:

$$\frac{\partial \mathcal{L}_{ind}}{\partial z_i} = \sum_{k=1}^T w_k t_k [-C_k \sin(t_k z_i) + S_k \cos(t_k z_i)], \quad (10)$$

which matches the direction of the ECF squared-error gradient when C_k and S_k represent the global cosine and sine errors. During the forward pass, every sample computes its contribution independently of its neighbors, allowing mini-batch execution without requiring the full massive batch in memory.

3.3 Gradient and Statistic Accumulation

To mimic a baseline batch size $N = K \times B$, we process K mini-batches of size B and defer the optimizer step until the end of the window. During these K steps:

- The model weights are effectively frozen because the optimizer step is deferred.

- The active branch uses $\mathcal{L}_{ind}$ to accumulate gradients.
- The shadow branch accumulates raw characteristic-function sums under `no_grad`.

Because the model weights are frozen during the accumulation window, representation drift is minimized. The shadow statistic buffer therefore does not need to maintain a moving average in the usual online-learning sense. It can accumulate raw sums and counts:

$$S_{\cos}^{(shadow)}(t_k) \leftarrow S_{\cos}^{(shadow)}(t_k) + \sum_{j=1}^B \cos(t_k Z_{shadow,j}), \quad (11)$$

$$n^{(shadow)} \leftarrow n^{(shadow)} + B, \quad (12)$$

with an analogous update for $S_{\sin}^{(shadow)}(t_k)$. At the end of the window, these sums are normalized into the estimates used by the next active buffer:

$$\mathcal{H}_{\cos}^{(shadow)}(t_k) = \frac{S_{\cos}^{(shadow)}(t_k)}{n^{(shadow)}}, \quad (13)$$

and analogously for $\mathcal{H}_{\sin}^{(shadow)}(t_k)$. When all mini-batches have the same size, this is equivalent to a Cumulative Moving Average (CMA), but the implementation is more naturally described as sum-and-count accumulation.

For A projection axes and T frequency points, the shadow statistic buffer stores

$$S_{\cos}^{(shadow)}, S_{\sin}^{(shadow)} \in \mathbb{R}^{A \times T} \quad (14)$$

plus a scalar count. Thus its persistent memory is $O(AT)$, independent of the virtual batch size N and independent of the latent dimension D .

3.4 CMA, EMA, and Why Accumulation Is Preferred

The update rule can be viewed in three related ways:

- **Raw accumulation:** Store the cosine and sine sums plus their sample count, then divide once when the buffer is promoted. This is the cleanest formulation for gradient accumulation because it exactly matches the statistics of a virtual batch of size $N = K \times B$.
- **CMA:** Update a running normalized estimate with equal weight for every sample or equal-sized mini-batch. This is mathematically equivalent to raw accumulation, but it can obscure the fact that the method is building a finite-window virtual batch rather than tracking a drifting process.
- **EMA:** Weight recent mini-batches more heavily. This may be useful only when model weights or data distribution drift inside the accumulation window, but it no longer represents the exact characteristic function of the virtual batch. For the synchronized DB-SIGReg setting, EMA should be treated as an optional approximation rather than the default mechanism.

3.5 The Swap Mechanism

Upon reaching step K , the system performs one optimizer step using the accumulated gradients. Immediately afterward:

1. A_{active} is discarded.
2. A_{shadow} and its fully matured accumulated statistics $\mathcal{H}$ are promoted to become the new active state.
3. A new random matrix is generated to become the new A_{shadow} , with its sums and count initialized to zero.

This swap is mathematically seamless. The newly promoted active buffer has statistically valid normalized estimates computed from sums accumulated in its own projection frame, preventing gradient shock or loss spikes during the rotation of the projection axes.

4 Computational Complexity

The overhead separates into three parts. First, the persistent shadow statistic buffer is small: it stores two arrays of size $A \times T$ and a count, so it scales as $O(AT)$. Second, the shadow projection operator scales as $O(DA)$ if stored as a dense random matrix, or less if generated from a seed or implemented with a structured random transform. Third, the transient shadow activations scale as $O(BA)$ and are not retained for backpropagation because the branch runs under `no_grad`.

Given a latent dimension $D = 8192$ and $A = 1024$ projection directions, the active and shadow projections can be implemented as a single concatenated projection producing $(B \times D) \times (D \times 2A)$. On modern hardware accelerators, widening this terminal projection is far cheaper than storing the intermediate activations of a backbone encoder for a massive batch. The effective statistical batch size scales from B to $N = K \times B$, while the activation memory remains tied to the mini-batch size B .

5 Conclusion and Future Work

We have outlined a theoretical framework that bridges the gap between massive-batch regularization algorithms and the physical constraints of single-GPU or consumer hardware. By decoupling projection geometry through a double-buffered shadow update and replacing the coupled squared batch loss with an independent sample pseudo-loss, DB-SIGReg enables temporal distribution matching in high-dimensional spaces without succumbing to null space vulnerabilities.

[Experimental section and empirical validation to be added.]